

디지털 시스템 설계 APB 기반RGB LED 제어시스템 REPORT

2025. 06. 20

담당교수	전자공학부 이진
학과	전자공학부
학번	2022142046
이름	최다빈

1. 프로젝트 목표

본 프로젝트의 목표는 APB(Advanced Peripheral Bus) 인터페이스를 통해 PWM(Pulse Width Modulation) 기반의 RGB LED 컨트롤러를 직접 설계하고, 이를 MicroBlaze 프로세서 시스템에 통합하는 것입니다. 최종적으로는 PC의 UART 터미널을 통해 컬러 값을 입력받아 2개의 RGB LED를 독립적으로 제어할 수 있는 완전한 임베디드 시스템을 구현합니다. 각 LED의 R, G, B 채널은 0부터 255까지의 256단계 세기 조절이 가능하며 다채로운 색상 표현이 가능하도록 하는 것을 핵심 목표로 삼았습니다.

2. 개발 환경

타겟 보드: xc7s50csg324-1 FPGA 기반 보드 (Nexys A7-50T)

하드웨어 설계: Vivado Design Suite 2023.1

소프트웨어 개발: Vitis Unified Software Platform 2023.1

하드웨어 기술 언어: Verilog HDL , C

시리얼 통신: Tera Term (9600 bps)

3-1-2. 주소 맵 (Address Map)

MicroBlaze가 주변장치를 제어하기 위해 사용하는 메모리 맵은 다음과 같습니다. AXI to APB Bridge를 통해 커스텀 RGB 컨트롤러에 64KB의 주소 공간을 할당했습니다.

Address Editor						
Assigned (5) Unassigned (0) Excluded (0) Hide All						
Name	Interface	Slave Segment	Master Base Address	Range	Master High Address	
Network 0						
/microblaze_0						
/microblaze_0/Data (32 address bits : 4G)						
/APB_M_0	APB_M_0	Reg	0x44A0_0000	64K	0x44A0_FFFF	
/axi_uartlite_0/S_AXI	S_AXI	Reg	0x4060_0000	64K	0x4060_FFFF	
/microblaze_0_axi_intc/S_AXI	s_axi	Reg	0x4120_0000	64K	0x4120_FFFF	
/microblaze_0_local_memory/dlmb_bram_if_cntlr/SLMB	SLMB	Mem	0x0	128K	0x1_FFFF	
Network 1						
/microblaze_0						
/microblaze_0/Instruction (32 address bits : 4G)						
/microblaze_0_local_memory/ilmb_bram_if_cntlr/SLMB	SLMB	Mem	0x0	128K	0x1_FFFF	

그림2. 주요 주변장치 주소 맵

3-1-3. RGB Control Register정보

소프트웨어가 커스텀 RGB 컨트롤러의 각 채널을 개별적으로 제어할 수 있도록, 다음과 같이 RGB Control Register 정보를 설계했습니다. 각 레지스터는 8비트 폭을 가지며, MicroBlaze가 APB 브릿지를 통해 32비트 단위로 접근합니다.

주소 오프셋	레지스터 이름	접근	설명
0x00	REG_RGB0_R	R/W	RGB LED 0번의 Red 채널 밝기 값 (0~255)
0x04	REG_RGB0_G	R/W	RGB LED 0번의 Green 채널 밝기 값 (0~255)
0x08	REG_RGB0_B	R/W	RGB LED 0번의 Blue 채널 밝기 값 (0~255)
0x0C	REG_RGB1_R	R/W	RGB LED 1번의 Red 채널 밝기 값 (0~255)
0x10	REG_RGB1_G	R/W	RGB LED 1번의 Green 채널 밝기 값 (0~255)
0x14	REG_RGB1_B	R/W	RGB LED 1번의 Blue 채널 밝기 값 (0~255)

그림3. RGB 컨트롤러 레지스터 맵

3-1-3. PWM 생성 로직 구현(rgb_pwm_controller.v)

APB 인터페이스를 통해 설정된 Duty Cycle 값을 바탕으로 실제 PWM 파형을 생성합니다.

AXI to APB Bridge를 통해 전달되는 APB 버스 신호(PADDR, PSEL, PENABLE, PWRITE 등)를 해석

```
//=====
// 2. APB Write Logic - Register Storage
//=====
reg [7:0] r_duty_reg [0:5]; // Array of 6 registers for each PWM channel's duty cycle

// APB Write Transaction Signal
wire w_apb_write_en = PSEL & PENABLE & PWRITE;

always @(posedge PCLK or negedge PRESETn) begin
    if (!PRESETn) begin
        // Reset all duty registers to 0
        r_duty_reg[0] <= 8'h00;
        r_duty_reg[1] <= 8'h00;
        r_duty_reg[2] <= 8'h00;
        r_duty_reg[3] <= 8'h00;
        r_duty_reg[4] <= 8'h00;
        r_duty_reg[5] <= 8'h00;
    end else begin
        if (w_apb_write_en) begin
            // Decode address and write data to the corresponding register
            // We only look at PADDR[4:2] because addresses are 4-byte aligned
            case (PADDR[ADDR_WIDTH-1:2])
                ADDR_LED0_R_DUTY: r_duty_reg[0] <= PWDATA[7:0];
                ADDR_LED0_G_DUTY: r_duty_reg[1] <= PWDATA[7:0];
                ADDR_LED0_B_DUTY: r_duty_reg[2] <= PWDATA[7:0];
                ADDR_LED1_R_DUTY: r_duty_reg[3] <= PWDATA[7:0];
                ADDR_LED1_G_DUTY: r_duty_reg[4] <= PWDATA[7:0];
                ADDR_LED1_B_DUTY: r_duty_reg[5] <= PWDATA[7:0];
                default:           ; // Do nothing for undefined addresses
            endcase
        end
    end
end
```

쓰기 동작

PSEL과 PENABLE, 그리고 PWRITE 신호가 모두 활성화되면, 소프트웨어가 쓰기 동작을 요청한 것으로 판단합니다.

이때 PADDR 주소를 디코딩하여 목표 레지스터를 선택하고, PWDATA 버스에 실려온 데이터를 해당 레지스터에 저장합니다.

읽기 동작

PSEL과 PENABLE은 활성화되고 PWRITE가 비활성화되면, 읽기 동작으로 판단합니다.

PADDR 주소에 해당하는 레지스터 값을 PRDATA 버스로 출력하여 소프트웨어에 전달합니다.

소프트웨어로부터 받은 각 RGB 채널의 밝기 값을 저장하는 6개의 8비트 레지스터 집합

```
//=====
// 2. APB Write Logic - Register Storage
//=====
reg [7:0] r_duty_reg [0:5]; // Array of 6 registers for each PWM channel's duty cycle
```

레지스터들은 PWM Generator가 실시간으로 참조하는 '설정값 저장소' 역할을 합니다.

기준 타이머 생성(PWM동작)

```
//=====
// 4. PWM Generation Logic
//=====
// PWM Period Counter: ~1ms cycle for 100MHz clock (100,000 cycles)
localparam PWM_PERIOD = 100000;
reg [16:0] pwm_period_counter; // Needs to hold up to 99,999 (17 bits)

always @(posedge PCLK or negedge PRESETn) begin
    if (!PRESETn) begin
        pwm_period_counter <= 0;
    end else begin
        if (pwm_period_counter < PWM_PERIOD - 1) begin
            pwm_period_counter <= pwm_period_counter + 1;
        end else begin
            pwm_period_counter <= 0;
        end
    end
end
end
```

0부터 99,999까지 1씩 증가하고 다시 0으로 돌아가는 17비트 카운터(pwm_period_counter)를 설계하여 1ms의 기준 주기를 생성합니다 (100,000 클럭 * 10ns/클럭 = 1ms).

PWM(펄스 폭 변조)은 '1'이 유지되는 시간의 비율(듀티 사이클)을 조절하여 LED의 밝기와 같은 아날로그적인 효과를 내는 디지털 신호입니다.

밝기 값 비교 및 PWM 신호 생성

```
// Generate 6 PWM channels
genvar i;
generate
  for (i = 0; i < 6; i = i + 1) begin : pwm_channel_gen
    // Scale the 8-bit duty value (0-255) to the PWM period (0-99999)
    // Scaling factor: 100000 / 256 ~ 390.625. We use 391 for simplicity.
    wire [16:0] w_scaled_duty;
    assign w_scaled_duty = r_duty_reg[i] * 17'd391;

    // Generate PWM output signal based on comparison
    // Handle edge cases: duty=255 is always ON, duty=0 is always OFF.
    assign o_pwm_ch[i] = (r_duty_reg[i] == 8'hFF) ? 1'b1 :
                        (r_duty_reg[i] == 8'h00) ? 1'b0 :
                        (pwm_period_counter < w_scaled_duty);

  end
endgenerate
```

endmodule

소프트웨어로부터 받은 8비트 밝기 값(0~255)을 1ms 주기에 맞게 스케일링한 후, 주기 카운터와 실시간으로 비교하여 PWM 신호를 생성합니다.

이 로직의 핵심은 `pwm_period_counter < w_scaled_duty` 비교문입니다. 주기 카운터의 값이 스케일링된 듀티 값보다 작은 동안만 해당 PWM 채널의 출력을 High(1)로 설정하고, 그 외의 시간에는 Low(0)로 설정합니다. 이를 통해 레지스터에 저장된 값에 비례하는 펄스 폭을 가진 PWM 신호가 생성되어, 최종적으로 LED의 밝기를 256단계로 제어하게 됩니다.

3-2. vitis

```
//=====
// 7. Command Processing Function
//=====
void process_command(char *cmd_buffer) {
    char *argv[5]; // Max 4 arguments + 1 for safety
    int argc = 0;
    char *token;

    // Parse the command string by spaces using strtok
    token = strtok(cmd_buffer, " ");
    while (token != NULL && argc < 5) {
        argv[argc++] = token;
        token = strtok(NULL, " ");
    }

    if (argc == 0) {
        return; // Empty command
    }
}
```

atoi 함수로 변환된 LED 번호를 기반으로 목표 레지스터의 정확한 주소를 계산한 후, 가장 중요한 Xil_In32 함수를 호출하여 APB 버스를 통해 해당 주소의 하드웨어 레지스터 값을 읽어옵니다.

이렇게 가져온 R, G, B 값들은 프로젝트 명세서에서 지정한 형식에 맞춰 xil_printf 함수로 터미널에 출력됩니다.

```

// Branch based on number of arguments
if (argc == 4) { // Case 1: Set Color (e.g., "0 100 20 50")
    int led_num = atoi(argv[0]);
    int r_val = atoi(argv[1]);
    int g_val = atoi(argv[2]);
    int b_val = atoi(argv[3]);

    // Input validation
    if ((led_num != 0 && led_num != 1) ||
        (r_val < 0 || r_val > 255) ||
        (g_val < 0 || g_val > 255) ||
        (b_val < 0 || b_val > 255)) {
        xil_printf("Error: Invalid arguments. Use: (LED# 0-1) (R G B 0-255)\r\n");
        return;
    }

    // Calculate register addresses and write values
    u32 base_offset = (led_num == 0) ? 0 : (REG_OFFSET_LED1_R);
    Xil_Out32(PWM_CTRL_BASEADDR + base_offset + REG_OFFSET_LED0_R, r_val);
    Xil_Out32(PWM_CTRL_BASEADDR + base_offset + REG_OFFSET_LED0_G, g_val);
    Xil_Out32(PWM_CTRL_BASEADDR + base_offset + REG_OFFSET_LED0_B, b_val);

    xil_printf("OK: LED %d set to R:%d, G:%d, B:%d\r\n", led_num, r_val, g_val, b_val);
}

```

atoi 함수로 변환된 LED 번호와 R, G, B 값을 기반으로 목표 레지스터의 정확한 주소를 계산한 후, 가장 중요한 Xil_Out32 함수를 호출하여 APB 버스를 통해 해당 주소의 하드웨어 레지스터 값을 씁니다. 이 동작은 직접적으로 하드웨어의 상태를 변경하여, PWM 출력 파형을 바꾸고 최종적으로 LED의 색상을 변화시킵니다.

```

} else if (argc == 1) { // Case 2: Get Status (e.g., "0")
    int led_num = atoi(argv[0]);

    if (led_num != 0 && led_num != 1) {
        xil_printf("Error: Invalid LED number. Use 0 or 1.\r\n");
        return;
    }

    // Calculate register addresses and read values
    u32 base_offset = (led_num == 0) ? 0 : (REG_OFFSET_LED1_R);
    int r_val = Xil_In32(PWM_CTRL_BASEADDR + base_offset + REG_OFFSET_LED0_R);
    int g_val = Xil_In32(PWM_CTRL_BASEADDR + base_offset + REG_OFFSET_LED0_G);
    int b_val = Xil_In32(PWM_CTRL_BASEADDR + base_offset + REG_OFFSET_LED0_B);

    // Print in the specified format
    xil_printf("RGB %d : R-%d, G-%d, B-%d\r\n", led_num, r_val, g_val, b_val);
}

```

제어할 때와 마찬가지로 목표 레지스터의 주소를 계산한 뒤, 가장 중요한 Xil_In32 함수를 호출하여 APB 버스를 통해 해당 주소의 하드웨어 레지스터 값을 읽어옵니다.

이렇게 가져온 R, G, B 값들은 프로젝트 명세서에서 지정한 형식에 맞춰 xil_printf 함수로 터미널에 출력됩니다.

4. 전체 프로그램의 흐름도

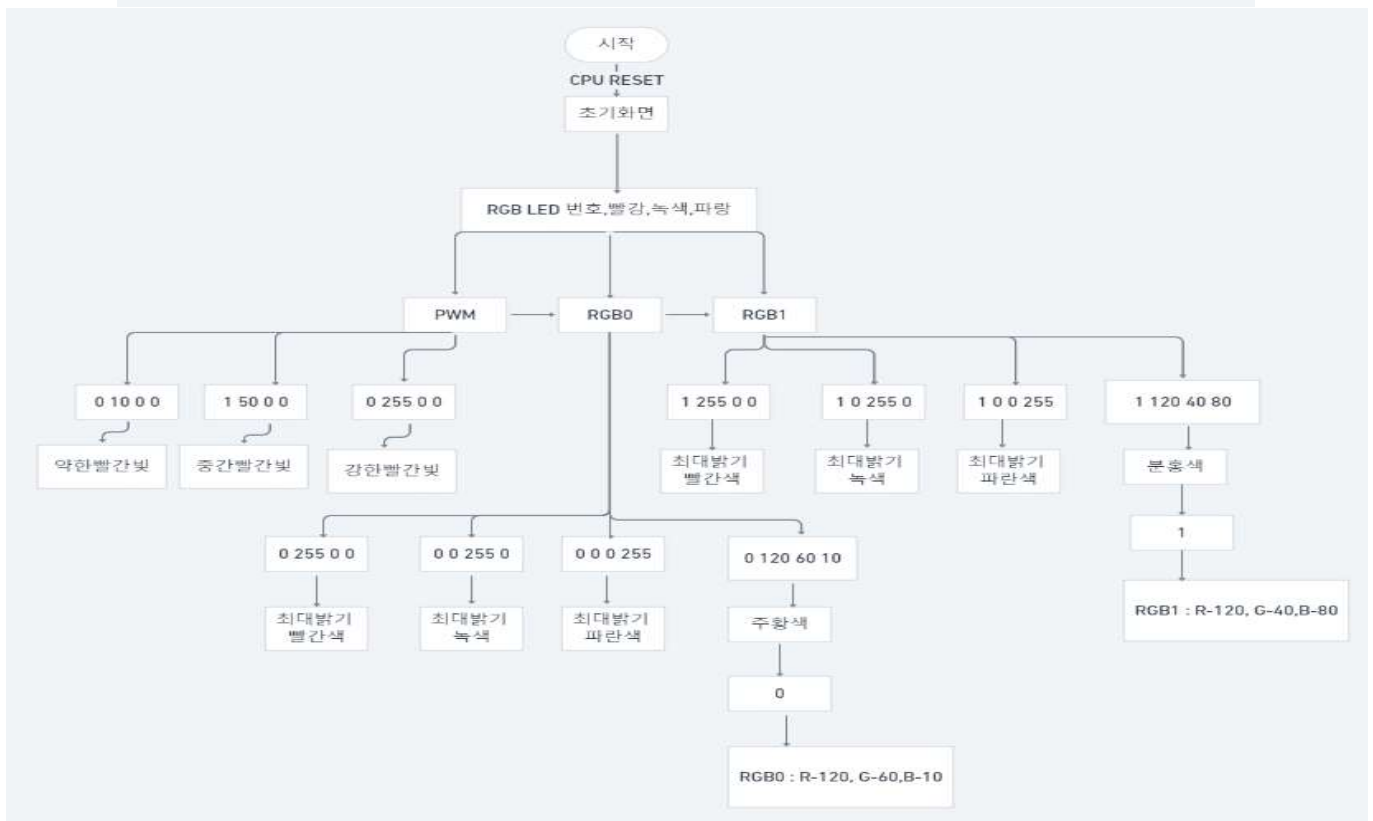
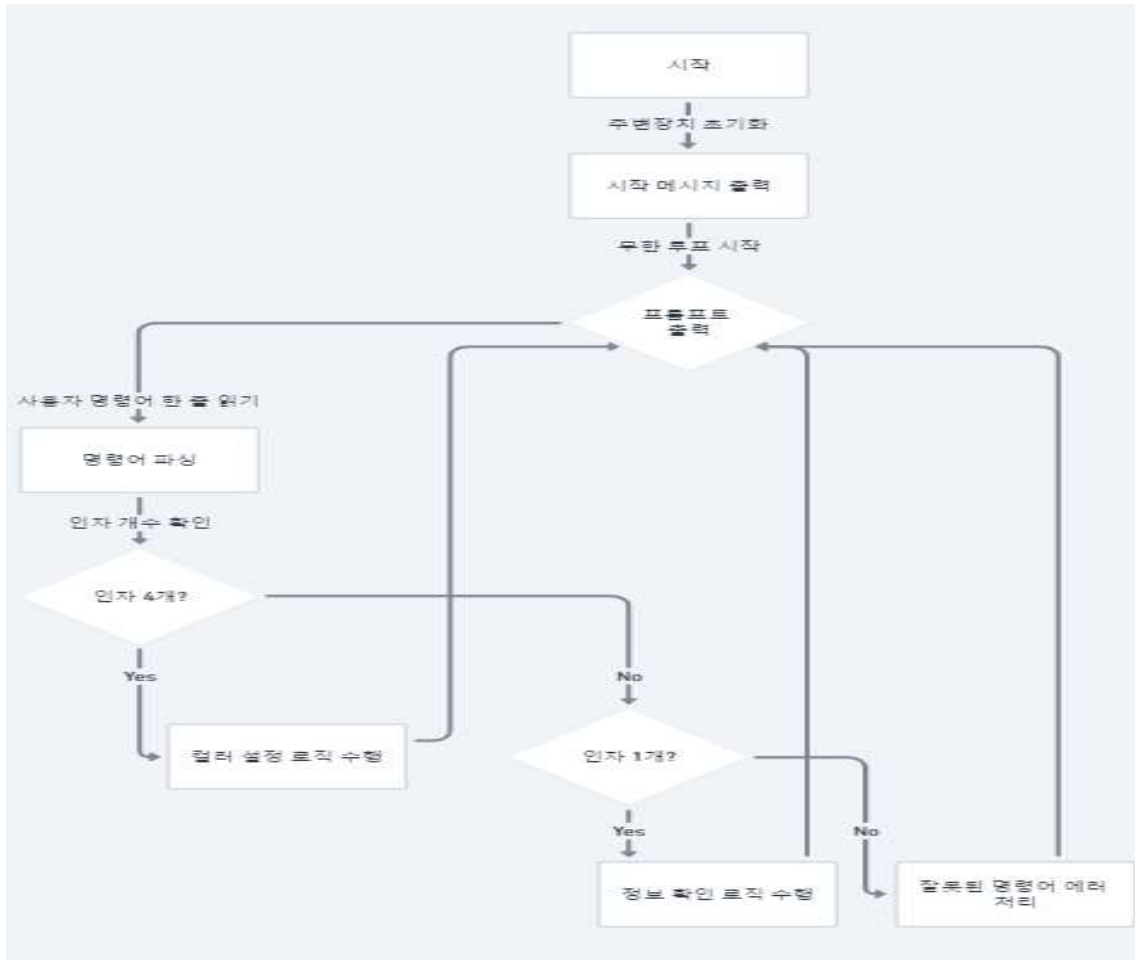


그림4. 전체 흐름도와 과제 내용 그림

5. 명령어 체계 설명

소프트웨어는 다음과 같은 두 가지 명확한 명령어 체계를 가집니다.

명령어 종류	입력 형식 (Syntax)	예시	설명
컬러 설정	(LED 번호) (R값) (G 값) (B값)	0 120 60 10	지정된 LED 번호(0 또는 1)의 R, G, B 레지스터에 해당 값을 써 서 컬러를 변경합니다.
정보 확인	(LED 번호)	1	지정된 LED 번호(0 또는 1)의 R, G, B 레지스터 값을 읽어와 지 정된 형식으로 출력합니다.

그림5 . 소프트웨어 명령어 체계

명령 입력 시 어떤 동작이 수행되는지를 나타냅니다.

테스트 항목	터미널 입력 명령어	검증 내용 (Expected Behavior)	확인
1. 초기화	(전원 인가)	- 터미널에 "RGB LED Control Test Program" 출력 - RGB 0, RGB 1 LED 모두 꺼진 상태 확인	O
2. PWM 동작	0 10 0 0	- RGB 0에 약한 빨간 빛이 나오는지 확인	O
	1 50 0 0	- RGB 1에 빨간 빛이 RGB 0보다 밝은지 확인	O
	0 255 0 0	- RGB 0에 빨간 빛이 RGB 1보다 밝은지 확인	O
3. RGB 0 컬러 표현	0 255 0 0	- RGB 0에 최대 밝기의 빨간색 점등 확인	O
	0 0 255 0	- RGB 0에 최대 밝기의 녹색 점등 확인	O
	0 0 0 255	- RGB 0에 최대 밝기의 파란색 점등 확인	O
	0 120 60 10	- RGB 0에 주황색 톤의 빛 점등 확인	O
	0	- 터미널에 "RGB 0 : R-120, G-60, B-10" 출력 확인	O
4. RGB 1 컬러 표현	1 255 0 0	- RGB 1에 최대 밝기의 빨간색 점등 확인	O
	1 0 255 0	- RGB 1에 최대 밝기의 녹색 점등 확인	O
	1 0 0 255	- RGB 1에 최대 밝기의 파란색 점등 확인	O
	1 120 40 80	- RGB 1에 분홍색 톤의 빛 점등 확인	O
	1	- 터미널에 "RGB 1 : R-120, G-40, B-80" 출력 확인	O

그림 6. 소프트웨어 명령입력에 따른 동작수행

6. 느낀점 및 개선점

6-1. 배운점

이번 기말고사 대체 프로젝트를 통해 단순한 IP 활용을 넘어, APB라는 표준 버스 프로토콜을 이해하고 이에 맞는 커스텀 IP를 직접 설계하여 MicroBlaze 시스템에 통합하는 전 과정을 경험할 수 있었습니다.

특히, Verilog로 PWM 생성기와 레지스터 파일을 설계하고, 이를 소프트웨어에서 메모리 맵 주소를 통해 직접 제어하는 과정은 하드웨어와 소프트웨어가 어떻게 긴밀하게 상호작용하는지 깊이 이해하는 데 큰 도움이 되었습니다.

6-2. 느낀점

교수님께서 강의에서 설명해주신 PWM의 원리와 사람의 시각적 인지 특성(300Hz 이상)을 고려하여 PWM 주파수를 1kHz로 설정하는 등, 이론적 지식을 실제 설계 사양에 반영하는 실용적인 경험을 할 수 있었습니다. 또한, Xil_Out32, Xil_In32와 같은 저수준 함수를 사용하여 메모리 맵에 직접 접근하는 방식은 드라이버 뒤에 숨겨진 실제 하드웨어 제어 방식을 엿볼 수 있는 좋은 기회였습니다.

6-3. 부족한 점

1. PWM 생성 방식의 정확성 문제

부족한 점

현재 코드는 8비트 듀티 값(0-255)을 17비트 주기 카운터(0-99999)에 맞게 스케일링하기 위해, 근사값인 391을 곱하고 있습니다 ($100000 / 256 \approx 390.625$).

이 방식은 구현이 간단하지만, 선형적인 밝기 변화를 보장하지 못하며 미세한 오차를 누적시킬 수 있습니다. 예를 들어, r_duty_reg 값이 1일 때와 2일 때의 실제 펄스 폭 차이가 391 클럭 사이클이 되어, 의도한 1/256 단계보다 더 큰 폭으로 변하게 됩니다. 또한, $255 * 391 = 99705$ 이므로, 듀티 값이 255일 때 펄스 폭이 100%가 되지 못하고 약 99.7%에 머무릅니다.

2. 레지스터 파일의 확장성 및 안정성 부족

부족한 점

현재 레지스터 읽기/쓰기 로직은 case 문을 사용하여 각 주소에 대해 하드코딩되어 있습니다. 만약 레지스터 개수가 늘어나면 코드를 일일이 수정해야 합니다.

APB 쓰기 동작이 PENABLE 신호에만 의존하여 한 클럭 사이클 내에 완료된다고 가정합니다. 만약 내부 로직이 복잡해져 여러 사이클이 필요하다면, 현재 구조로는 데이터가 올바르게 쓰이는 것을 보장할 수 없습니다.

6-4. 향후 개선 가능성

1. PWM 생성 방식의 정확성 문제

개선 방안

Bresenham's line algorithm과 유사한 디지털 미분 분석기(Digital Differential Analyzer, DDA) 원리를 적용할 수 있습니다. 100MHz 클럭마다 `r_duty_reg` 값을 누산기(accumulator)에 더하고, 누산기에서 오버플로우가 발생할 때만 PWM 출력을 토글시키는 방식입니다. 이 방법은 곱셈기 없이 덧셈기만으로 훨씬 더 정확하고 선형적인 PWM 생성이 가능합니다.

또는, 더 큰 비트의 카운터와 비교기를 사용하여 $r_duty_reg * PWM_PERIOD / 256$ 과 같은 연산을 통해 이상적인 듀티 값을 계산할 수 있습니다. 이는 더 많은 로직 리소스를 사용하지만 직관적이고 정확합니다.

2. 레지스터 파일의 확장성 및 안정성 부족

개선 방안

레지스터 파일 부분을 파라미터화(parameterize) 하여 레지스터 개수나 폭이 변경되어도 코드가 자동으로 확장될 수 있도록 설계할 수 있습니다.

읽기/쓰기 동작을 상태 머신(FSM)으로 구현하여, PSEL이 들어오면 IDLE 상태에서 DECODE, ACCESS 등의 상태를 거치도록 설계하면 더 복잡하고 안정적인 레지스터 접근이 가능합니다. 비록 이 프로젝트에서는 PREADY를 1로 고정하여 단순화했지만, 이는 확장성을 저해하는 요소입니다.

번호	테스트 내용	동작 확인	동영상 시점 (mm:ss)
1	초기화	0	00:01
2	PWM 동작	0	00:05
3	RGB 0 컬러 표현	0	00:39
4	RGB 1 컬러 표현	0	01:32

동영상파일첨부

<https://youtube.com/shorts/Qn91vLUlbw>